

opposite ends	<code>l, r := 0, len(s)-1; move the one that must move</code>
pair sum in a SORTED slice: <code>sum < target -> l++, else r--</code>	
palindrome check	
container-with-most-water	
same direction (read/write) in-place filter and dedup:	
<code>w := 0</code>	
<code>for r := range s { if keep(s[r]) { s[w] = s[r]; w++ } }</code>	
<code>s = s[:w]</code>	
Dutch national flag	three pointers, one pass, three partitions
fast/slow	cycle detection, middle of a list

SLIDING WINDOW

```
fixed size k          add s[r], and when r >= k, remove s[r-k]
variable size         grow r; while the window is invalid, shrink l
  for r := range s {
    add(s[r])
    for !valid() { remove(s[l]); l++ }
    best = max(best, r-l+1)
  }
longest substring w/o repeats  window + a map of last-seen indices
min window covering           window + a need/have counter map
(any "contiguous subarray/substring" question is one of these two shapes)
```

MAPS AS SETS & COUNTERS

```
seen := map[T]struct{}{}    the set; seen[v] = struct{}{}
_, ok := seen[v]           contains
count := map[T]int{}        the frequency counter; count[v]++
group := map[K][]T{}        group-by; group[k] = append(group[k], v)
two-sum                    map of value -> index, one pass
group anagrams              sorted-letters as the key
(a map turns most  $O(n^2)$  scans into  $O(n)$  – that's the whole trick)
```

HEAPS: the mental model

```
a heap IS an array          no pointers; the tree is implicit
parent of i                 (i-1)/2
children of i               2i+1 and 2i+2
min-heap property           every parent <= its children (max-heap: >=)
NOT sorted                  only the ROOT is guaranteed
sift up (after push)        swap with the parent while it's out of order –  $O(\log n)$ 
sift down (after pop)       move the last element to the root, sink it –  $O(\log n)$ 
heapify                     sift down from  $n/2-1$  to  $0$  –  $O(n)$ , not  $O(n \log n)$ 
peek                          $h[0]$  –  $O(1)$ 
```

container/heap

```
implement heap.Interface    sort.Interface (Len, Less, Swap) PLUS:
  func (h *H) Push(x any)   { *h = append(*h, x.(T)) }
  func (h *H) Pop() any     { old := *h; n := len(old)
                             v := old[n-1]; *h = old[:n-1]; return v }
CALL THE PACKAGE FUNCTIONS  heap.Push(h, v) / heap.Pop(h) – NEVER h.Push/h.Pop
                             (yours only touch the slice; the package's fix the order)
heap.Init(h)                 $O(n)$  heapify of an existing slice
heap.Fix(h, i)               after changing an element's priority in place
heap.Remove(h, i)            delete at an index
Less decides min or max       $a < b \rightarrow$  min-heap;  $a > b \rightarrow$  max-heap
priority queue               store an index field on the item so Fix/Remove work
```

WHAT HEAPS ARE FOR

top-K / k-th largest	keep a MIN-heap of size k $\rightarrow O(n \log k)$, not $O(n \log n)$
merge k sorted lists	a heap of the k current heads
running median	a max-heap of the low half + a min-heap of the high half, rebalanced so their sizes differ by at most 1
Dijkstra	the frontier, ordered by distance
scheduling / meeting rooms	a heap of end times
k closest points	a max-heap of size k by distance
Huffman coding	repeatedly pop the two smallest, push their sum

GRAPH REPRESENTATION

adjacency list	<code>map[T][]T</code> or <code>[][]int</code> – the default; $O(V+E)$ memory
adjacency matrix	<code>[][]bool</code> – $O(V^2)$; only for dense graphs or $O(1)$ edge tests
edge list	<code>[]Edge{From, To, Weight}</code> – for Kruskal and for input
directed vs undirected	undirected = add BOTH directions when you build it
weighted	<code>map[T][]Edge</code> , or a parallel weight map
implicit graph	no data structure at all: the neighbours are computed (word ladder, grid cells, state transitions)
grid as a graph	neighbours = the 4 (or 8) in-bounds cells

BFS & DFS

BFS (queue)	shortest path in an UNWEIGHTED graph
<pre>q := []T{start}; visited := map[T]bool{start: true} for len(q) > 0 { n := q[0]; q = q[1:] for _, m := range adj[n] { if !visited[m] { visited[m] = true parent[m] = n; q = append(q, m) } } }</pre>	
mark visited ON ENQUEUE	not on dequeue, or nodes enter the queue twice
path reconstruction	walk the parent map back from the target, then reverse
level by level	capture <code>len(q)</code> before the inner loop
multi-source BFS	seed the queue with EVERY source at distance 0
DFS (recursion or a stack)	reachability, cycles, topological order, components
recursive	visited map + recurse over neighbours
iterative	an explicit stack (and it visits in a different order)
connected components	loop over all nodes; every unvisited one starts a new BFS/DFS $\rightarrow$ that's one component
bipartite check	2-colour with BFS; a same-coloured edge means no

CYCLES, TOPOLOGICAL ORDER & UNION-FIND

directed cycle (3-colour)	white unvisited / grey on the current path / black done an edge to a GREY node is a back edge = a cycle
undirected cycle	DFS, ignore the edge you came from; any other visited neighbour is a cycle
topological sort (Kahn's)	in-degree array; queue every 0; on pop, decrement the neighbours' in-degrees and enqueue the new zeros fewer nodes emitted than exist $\rightarrow$ there is a cycle
topological sort (DFS)	post-order, then REVERSE
union-find (DSU)	<code>parent[] + rank[]</code> ; find with PATH COMPRESSION, union by rank $\rightarrow$ near $O(1)$ amortized cycle detection in undirected graphs, and Kruskal

WEIGHTED GRAPHS

Dijkstra	non-negative weights; a min-heap frontier
dist := map[T]int{start: 0}; push {start, 0}	
pop the smallest; SKIP if the popped distance is stale; relax the neighbours	
$O((V+E) \log V)$	and NO negative edges (use Bellman-Ford for those)
Bellman-Ford	[*] handles negative edges, detects negative cycles, $O(VE)$
Floyd-Warshall	[*] all-pairs shortest paths, $O(V^3)$
A*	[*] Dijkstra plus a heuristic
MST – Kruskal	sort the edges, add one if it joins two components (union-find decides) – $O(E \log E)$
MST – Prim	grow one tree with a heap frontier – $O(E \log V)$

TRAPS & MEMORIZE

h.Push instead of heap.Push	the invariant is never restored – silent wrong answers
forgetting heap.Init	on a pre-filled slice
Less with the wrong sign	a max-heap where you wanted a min-heap
marking visited on dequeue	duplicates in the queue, exponential blowup
BFS on a weighted graph	gives fewest EDGES, not the shortest distance
Dijkstra with negative edges	silently wrong; no error, no panic
mutating a slice while ranging	index carefully, or build a new one
sort.Search's predicate	must be monotone false->true or it returns nonsense
binary search on unsorted	no error, just a wrong answer
recursion depth on a big graph	use an explicit stack
undirected edges added once	half the graph is missing
comparing float weights with ==	accumulate error; compare with a tolerance